

Exécution du projet décomposition LU NF04

Github du projet : <https://github.com/elouan660/Projet-NF04-2026-LUd-composition>

Exécution en python :

Exemple du cours :

Programme exécuté selon l'exemple demandé en cours, fonctionne avec toutes valeurs et tailles de matrice carrées A comme demandé

```
A = [[2,3,1],[4,7,3],[6,18,5]] # Matrice recherchée  
b=[1,3,5]
```

Le programme affiche chaque étape de calcul

```
Matrice U:  
->Position 0-0  
2 0 0  
0 0 0  
0 0 0  
  
->Position 0-1  
2 3 0  
0 0 0  
0 0 0  
  
->Position 0-2  
2 3 1  
0 0 0  
0 0 0  
  
Matrice L:  
->Position 0-1  
1 0 0  
0 0 0  
0 0 0  
  
->Position 0-2  
1 0 0  
2 0 0  
0 0 0
```

Il affiche ensuite proprement les matrices L U, les vecteurs y, x et le vecteur résultat qui est une simple vérification, il est normalement toujours égal au vecteur b

```
Matrices L et U finales:
```

```
1 0 0
2 1 0
3 9 1
```

```
2 3 1
0 1 1
0 0 -7
```

```
vecteur y:
```

```
1.0
1.0
-7.0
```

```
vecteur x:
```

```
0.0
0.0
1.0
```

```
vérification du résultat:
```

```
1.0
3.0
5.0
```

```
Le résultat est correct
```

Autre exemple :

Exemple avec une matrice de taille différente

```
A = [[2,3,1,1],[4,7,3,1],[6,18,5,1],[1,1,1,1]] # Matrice recherchée
b=[1,3,5,1]
n=len(A)
```

```
Matrices L et U finales:
1 0 0 0
2 1 0 0
3 9 1 0
0 0 0 1

2 3 1 1
0 1 1 -1
0 0 -7 7
0 0 0 1

vecteur y:
1.0
1.0
-7.0
0.0
vecteur x:
0.0
0.0
1.0
0.0
vérification du résultat:
1.0
3.0
5.0
1.0
Le résultat est correct
```

On peut voir que le programme s'adapte parfaitement à la taille de la matrice

Exécution en C

Le programme en C suis le même algorithme qu'en python avec des adaptations mineures

```
Matrice U:  
->Position 0-0  
2 0 0  
0 0 0  
0 0 0
```

```
->Position 0-1  
2 3 0  
0 0 0  
0 0 0
```

```
->Position 0-2  
2 3 1  
0 0 0  
0 0 0
```

```
Matrice L:  
->Position 0-1  
1 0 0  
0 0 0  
0 0 0
```

```
Matrices L et U finales:
```

```
1 0 0  
2 1 0  
3 9 1
```

```
2 3 1  
0 1 1  
0 0 -7
```

```
vecteur y:  
1.000000  
1.000000  
-7.000000
```

```
vecteur x:  
0.000000  
0.000000  
1.000000
```

```
vérification du résultat:  
1.000000  
3.000000  
5.000000
```